hyperparameter Â¶

Implementation of the hyperparameter tuning using Optuna framework.

MODEL_PROJECT_TEMPLATE module-attribute Â¶

```
MODEL_PROJECT_TEMPLATE = 'ds_api_hyperparameter'
```

STUDY_NAME_TEMPLATE module-attribute Â¶

```
STUDY_NAME_TEMPLATE = 'ds_api_hyperparameter_study'
```

TRAINING_CONFIG_SCHEMA module-attribute Â¶

```
TRAINING_CONFIG_SCHEMA = Schema({'model_type': And(str, lambda value: any(model_type for model_t
ype in (__subclasses__()) if value in __name__)), 'model_parameters': And({Optional(str): Or(Or(s
tr, Number), And(list, lambda x: len(x) > 1), And({'type': Or('int', 'float', 'uniform', 'discre
te_uniform', 'loguniform'), 'range': And(list, lambda values: len(values) == 2, lambda values: r
equire_all_of_type(values, Number)), Optional('log', default=False): Or(True, False), Optional('
step'): int, Optional('q'): float}))}, lambda x: len(x) > 1)})
```

logger module-attribute Â¶

```
logger = getLogger(__name__)
```

DatabaseConnection dataclass Â¶

```
DatabaseConnection(database: str, user: str, password: str, host: str = 'localhost', port: int =
5432)
```

Validates a database connection to safely return a valid URI.

Currently only works with PostgreSQL DB.

database instance-attribute Â¶

```
database: str
```

host class-attribute instance-attribute Â¶

```
host: str = 'localhost'
```

password instance-attribute Â¶

```
password: str
```

port class-attribute instance-attribute Â¶

```
port: int = 5432
```

user instance-attribute Â¶

```
user: str
```

get_connection_uri Â¶

```
get_connection_uri() -> str
```

Builds the DB connection URI from a set of configs.

HyperparameterStudy Â¶

```
HyperparameterStudy(study_name: str, train_datasource: DataSource, validation_datasource: DataSource, target_field: str, target_value: str | int | bool = 'true', cost_field: str | None = None, model_project: ModelProject | None = None, optimization_metric: Metric = RECALL, target_metric: Metric = FALSE_POSITIVE_RATE, target_metric_value: float = 0.01, direction: StudyDirection = MAXIMIZE, database: DatabaseConnection | None = None, sampler: BaseSampler | None = None, pruner: BasePruner | None = None, load_if_exists: bool = True)
```

Bases: `Study`

A study corresponds to an optimization task, i.e., a set of trials.

Parameters:

Name	Type	Description	Default
`study_name` Â¶	`str`	Name given to the study.	*required*
`train_datasource` Â¶	`DataSource`	Datasource used for training models.	*required*
`validation_datasource` Â¶	`DataSource`	Datasource used for tuning parameters.	*required*
`target_field` Â¶	`str`	The name of the target field in the datasource.	*required*
`target_value` Â¶	`str or int or bool`	The target value in the datasource, defaults to "true".	`'true'`
`cost_field` Â¶	`str`	Cost field of the datasource. Definidng it will make the optimization cost based (optimizing for cost recall vs recall, for example).	`None`
`model_project` Â¶	`ModelProject`	The project to run the train and evaluation on. If not provided, one is created.	`None`
`optimization_metric` Â¶	`Metric`	Metric used to maximize/minimize. Default is recall.	`RECALL`
`target_metric` Â¶	`Metric`	Metric to pin while optuna maximizes/minimizes the optimization metric. Default is False Positive Rate.	`FALSE_POSITIVE_RATE`
	`float`	Value to aim for in the target metric. Default is 0.01.	`0.01`

Name	Type	Description	Default
target_metric_value Â¶			
direction Â¶	StudyDirection	Direction of optimization. Default is StudyDirection.MAXIMIZE	MAXIMIZE
database Â¶	DatabaseConnection	Connection object to store the studies data in. If not provided, in-memory storage is used, and the Study will not be persistent.	None
sampler Â¶	BaseSampler	A sampler object that implements background algorithm for value suggestion.	None
pruner Â¶	BasePruner	A pruner object that decides early stopping of unpromising trials.	None
load_if_exists Â¶	bool	Flag to control the behavior to handle a conflict of study names. Defaults to True, loading a previous study.	True

Examples:

```
 >>> app : App
>>> train_datasource = app.pullByName("train")
>>> validation_datasource = app.pullByName("validation")
>>> model_project = ModelProject.create(app, "model_project")
>>> database_connection = DatabaseConnection(
...     database="test",
...     user="my_user",
...     password="my_password"
... )
>>>
>>> study = HyperparameterStudy(
...     study_name="test_study",
...     train_datasource=train_datasource,
...     validation_datasource=validation_datasource,
...     target_field="label",
...     target_value="true",
...     cost_field="amount",
...     model_project=model_project,
...     optimization_metric=Metric.RECALL,
...     target_metric=Metric.FALSE_POSITIVE_RATE,
...     target_metric_value=0.01,
...     database=database_connection,
... )
```

run Â¶

```
 run(config: Path, ignored_fields: set | None = None, n_trials: int = 100, timeout: float | None
= None, n_jobs: int = 1, catch: tuple[Type[Exception], ...] = (), show_progress_bar: bool =
False, keep_trained_models: bool = False) -> None
```

Optimizes the study. Use this method instead of Study#optimize.

Parameters:

Name	Type	Description	Default
	Path		*required*

Name	Type	Description	Default
`config` Â¶		The model configuration file.	
`ignored_fields` Â¶	`Set`	Set of fields to be ignored in the model training.	`None`
`n_trials` Â¶	`int`	The number of trials. If not provided, it runs 100 trials. If timeout is provided the study will finish when one of the conditions is met. It can also be stopped if it receives a termination signal such as Ctrl+C or SIGTERM.	`100`
`timeout` Â¶	`float`	Stops study after the given number of seconds.	`None`
`n_jobs` Â¶	`int`	The number of parallel jobs. If this argument is set to -1, the number is set to CPU count.	`1`
`catch` Â¶	`tuple of type Exception`	A study continues to run even when a trial raises one of the exceptions specified in this argument. The dafault is to stop at any exception raised.	`()`
`show_progress_bar` Â¶	`bool`	Flag to show progress bars or not. Currently, progress bar is experimental feature and disabled when n_jobs > 1.	`False`
`keep_trained_models` Â¶	`bool`	Flag to control if trained models should be kept in Pulse or removed. By default, they are deleted after the evaluation and metrics calculation.	`False`

Raises:

Type	Description
`RuntimeError:`	If nested invocation of this method occurs.
`Example of yaml Random Forest Config to tune only the number of trees:`	
`` ```yaml ````	
`model_type: RandomForestConfig`	
`model_parameters:`	num_trees: type: int range: [50, 500] log: False
`` ``` ````	

Examples:

```
 >>> study : HyperparameterStudy
>>> study.run(
...     config=Path("random_forest_config.yml"),
...     ignored_fields={"card", "mcc"},
...     n_trials=100,
...     n_jobs=4,
... )
```